

A Reproduction Study of GuP for Exact Subgraph Matching

YI ZHA, Tsinghua University, China

JINPENG WANG, Tsinghua University, China

This paper presents a reproduction study of GuP, an exact subgraph matching method based on guard-based pruning. We implemented a correctness-oriented baseline matcher and extended it with a *GUP-lite* design that supports reservation-based pruning, nogood-based pruning, metric collection, and filtering-strategy ablation. The study includes a literature taxonomy, a clear statement of reproduction scope, experiments on three successful datasets (Yeast, Human, and HPRD), three optional extensions—query-result reuse, incremental result maintenance under single-edge data-graph updates, and subgraph homomorphism—and a time-boxed boundary evaluation on WordNet and Patents. Reservation-based pruning is the main source of reduction on the six fixed dense-4 workloads, while post-fix dense-8 validation shows that nogood reuse can become dominant when many histories induce the same failing future subproblem. The Python implementation still shows a clear scalability boundary.

1 Introduction

Exact subgraph matching is a fundamental graph query problem with applications in social-network analysis, biological interaction analysis, fraud detection, and graph database systems. Given a large vertex-labeled data graph and a smaller query graph, the goal is to enumerate all injective mappings that preserve both labels and edges. Because the search space grows rapidly with query size, practical systems rely heavily on candidate filtering, matching-order optimization, and state-aware pruning.

This paper studies *GuP*, a recent exact subgraph matching method based on guard-based pruning [1]. GuP introduces two complementary pruning ideas: reservation guards, which propagate injectivity-related constraints to earlier search steps, and nogood guards, which attempt to reuse dead-end information discovered during the search. These mechanisms are designed to reduce futile recursive exploration while preserving exactness.

The study combines a literature review, a focused reproduction of GuP, experiments on multiple benchmark datasets, an ablation of the two guard mechanisms, and a small extension on result reuse across related queries. Together, these parts make it possible to examine both the core pruning ideas of GuP and their behavior on real workloads.

1.1 Implementation Overview

The implementation consists of a baseline depth-first backtracking matcher and a second matcher that adds GuP-inspired reservation and nogood guards on top of the same search framework. The code is written in Python and emphasizes correctness, interpretability, and direct comparison across variants.

The remainder of the paper is organized as follows. Section 2 reviews the literature and positions GuP within a multi-dimensional taxonomy of exact subgraph matching methods. Sections 3–5 define the problem, summarize GuP, and describe the implementation. Sections 6–8 present the main experiments and the ablation study. Section 9 reports the result-reuse extension for related queries. Section 12 concludes the paper.

2 Literature Review and Multi-dimensional Taxonomy

Exact subgraph matching has been studied for more than one research generation, and representative progress has appeared from both the database and graph-algorithm communities. Prior work does not advance along a single axis:

Authors' Contact Information: Yi Zha, Tsinghua University, Beijing, China, chay22@mails.tsinghua.edu.cn; Jinpeng Wang, Tsinghua University, Beijing, China, wjp21@mails.tsinghua.edu.cn.

some methods emphasize aggressive candidate-space reduction before backtracking begins, some optimize the search order to expose contradictions earlier, some reuse dead-end information to avoid repeating equivalent failures, and others redesign the whole execution pipeline around candidate representation and set operations [1–6]. A useful review therefore needs a *multi-dimensional taxonomy* that explains how different methods trade off filtering power, search overhead, and implementation complexity, and how GuP fits into this larger design space.

2.1 Overview of Prior Work

Prior work on exact subgraph matching is highly diverse. The taxonomy below highlights the dominant optimization ideas behind representative methods and clarifies how GuP relates to them.

2.2 Taxonomy Dimensions

We classify representative work using four main dimensions.

- **Candidate filtering:** how aggressively a method shrinks the domain of each query vertex before or during the search. Typical signals include labels, degrees, neighborhood signatures, and local structural consistency. Stronger filtering reduces the branching factor, but it may require heavier preprocessing or more complex auxiliary data structures [3, 5].
- **Matching order:** how the method chooses the next query vertex to expand. Since exact matching is usually realized as state-space search, the order of expansion strongly affects whether contradictions appear early or late. Static heuristics are simple and cheap, while adaptive order selection can better react to the current partial embedding but also introduces additional control overhead [2, 5].
- **Conflict reuse / failing sets:** whether the method memorizes information learned from dead ends and reuses it to skip equivalent or dominated branches later. This dimension is especially important when many branches fail for similar reasons; in such cases, remembering the “shape” of failure may be more valuable than further tightening local consistency checks [1, 2].
- **System-level acceleration:** whether the method relies on heavier indexing, candidate-space engineering, specialized set operations, or a holistic execution framework beyond local pruning logic. This dimension separates algorithms that are conceptually lightweight but search-aware from methods whose performance depends heavily on broader system design [3, 6].

These dimensions are complementary. A strong exact matcher often combines several of them, but different papers prioritize different components. The taxonomy therefore highlights each method’s dominant emphasis.

2.3 Representative Research Lines

Candidate-space-oriented filtering. An influential early database line focuses on reducing the candidate space as much as possible before expensive enumeration begins. GraphQL is a representative example: it integrates signature-based pruning and graph access methods so that many impossible matches can be removed early, before full backtracking search is triggered [3]. This line established an important intuition for later work: exact matching performance is often determined by how small and how accurate the candidate domains are. In practice, however, such methods can still struggle when local signatures are insufficiently discriminative, because substantial ambiguity remains and must be resolved by downstream search.

Table 1. A multi-dimensional taxonomy of representative exact subgraph matching studies.

Study / Method	Cand. filt.	Order	Conflict reuse	System support	Main emphasis	Relation to GuP
GraphQL [3]	Strong	Moderate	No	Yes	Signature-based pruning and graph access methods for reducing candidate space early	Earlier candidate-space-oriented line
DAF [2]	Strong	Strong	Strong	Moderate	Dynamic programming, adaptive matching order, and failing sets in one exact matcher	Closest classical conflict-aware reference
In-depth study [5]	Covers many	Covers many	Covers many	Covers many	Comparative analysis showing which components dominate performance across workloads	Evaluation vocabulary and taxonomy support
RapidMatch [6]	Strong	Strong	Limited	Strong	Holistic high-performance query processing instead of a single pruning trick	Stronger system emphasis than GuP
VEQ [4]	Moderate	Moderate	Limited	Moderate	Equivalence-aware reduction of redundant matching work	Alternative redundancy-reduction line
GuP [1]	Moderate	Moderate	Strong	Moderate	Guard-based pruning through reservation and nogood mechanisms	Our reproduction target

Integrated search optimization. A second line treats exact subgraph matching explicitly as a search problem whose major components should be optimized together. DAF is representative of this direction because it combines dynamic programming, adaptive matching order, and failing sets within one exact matcher [2]. The importance of DAF in our review is twofold. First, it shows that search order is not a secondary heuristic but a first-class performance factor: different orderings can expose conflicts at very different depths. Second, it demonstrates that failure information can be summarized and reused, rather than discarded after each dead end. Compared with earlier candidate-space-oriented methods, this line moves more of the optimization effort from static preprocessing to dynamic search control.

Comparative understanding of in-memory systems. The field has also developed a more reflective line of work that compares existing exact matchers systematically. The in-depth study by Sun and Luo is especially valuable in this regard [5]. It analyzes how candidate generation, matching order, intermediate representation, and implementation decisions interact across workloads, instead of attributing performance to a single pruning idea. This perspective is useful here because it provides a vocabulary for discussing why a method is fast or slow, and it motivates reporting search-space indicators such as the number of partial mappings alongside runtime.

Holistic execution and system support. Another research line pushes exact matching toward a more holistic execution-engine perspective. RapidMatch is representative here: it redesigns subgraph query processing around the coordinated treatment of candidate generation, query execution, and intermediate-state handling [6]. This line is important because state-of-the-art performance often depends on the organization of the entire execution pipeline as much as on local pruning logic. Such methods are also harder to emulate faithfully, since low-level engineering choices contribute substantially to the final speedup.

Redundancy and equivalence reduction. A closely related but conceptually distinct line seeks to reduce redundant search by recognizing when different branches correspond to equivalent or near-equivalent computational states. VEQ is a representative method in this category [4]. It shows that repeated work can be reduced through equivalence relationships among search states, in addition to stronger filtering and failure memorization. Compared with DAF-style failing sets, this direction emphasizes broader compression of duplicated work.

Relation among these lines. Taken together, these studies suggest that exact subgraph matching has evolved along several complementary directions. Candidate-space methods try to shrink the search problem early; search-centric methods optimize how the search tree is traversed; conflict-reuse methods try to avoid rediscovering the same failures; and system-oriented methods improve the execution pipeline as a whole. This perspective also prepares the later analysis of GuP’s reservation guard and nogood guard.

2.4 Positioning of GuP in the Taxonomy

Under the above taxonomy, GuP is best understood as a *search-based exact matcher with dynamic state-aware pruning*. Its main contribution is the introduction of guards that evaluate whether a candidate extension is likely to make the remaining search infeasible [1]. In particular, the reservation guard propagates future injectivity pressure earlier in the search, while the nogood guard attempts to reuse information from previously discovered dead ends. GuP therefore lies between classical candidate-space methods and stronger failing-set-style conflict-reuse methods: it still operates inside a backtracking enumeration framework, but it pushes pruning decisions closer to the dynamic search state.

This position makes GuP especially suitable for focused reproduction and analysis. Compared with GraphQL, GuP is less dependent on heavyweight offline candidate-space summaries; compared with DAF, it exposes its pruning ideas through relatively modular guards; compared with RapidMatch, it is less tied to full-stack system engineering; and compared with VEQ, its pruning contribution is easier to align with the course-required statistics on partial mappings and filtered partial mappings [1–4, 6]. In other words, GuP offers a favorable balance between research novelty, conceptual clarity, ablation friendliness, and implementability.

2.5 Implications for the Rest of This Report

This review suggests three points that matter throughout the paper. GuP belongs to the broader conflict-aware line of exact matching methods. The experiments should therefore consider end-to-end runtime together with search-space indicators that reflect how the guards affect enumeration. It is also useful to separate algorithmic effects from system-level ones when interpreting results from a Python implementation.

3 Problem Definition and Evaluation Metrics

Let the data graph be $G = (V_G, E_G, \ell_G)$ and the query graph be $Q = (V_Q, E_Q, \ell_Q)$, where vertices are associated with discrete labels. The exact subgraph matching problem asks for all injective mappings

$$f : V_Q \rightarrow V_G$$

such that:

- (1) label preservation holds, that is, $\ell_Q(u) = \ell_G(f(u))$ for every query vertex $u \in V_Q$, and
- (2) edge preservation holds, that is, $(u, u') \in E_Q$ implies $(f(u), f(u')) \in E_G$.

In our implementation, all graphs are undirected, vertex-labeled, and loop-free. The core reproduction does not consider edge labels. Subgraph homomorphism and dynamic graph updates are out of scope for the core reproduction but are addressed as optional extensions: Section 11 implements subgraph homomorphism under the augmented edge set E_G^+ , and Section 10 incrementally maintains the result set under single-edge insertions and deletions.

3.1 Required Metrics

The project specification requires four core metrics. We make the counting rules explicit so that all experimental variants are directly comparable.

- **Runtime (ms)**: wall-clock runtime of the matching procedure, excluding file-loading overhead.
- **Result mappings**: the number of complete valid embeddings returned by the matcher.
- **Partial mappings**: the number of successful partial extensions generated during the search.
- **Paper-filtered partial mappings**: the number of partial extensions pruned specifically by the reproduced paper’s filtering strategies.

The last metric requires special clarification. Our internal counter `pruned_partial_mappings` records *all* rejected extensions, including injectivity and edge-consistency conflicts. However, the assignment specifically asks for the number of partial mappings filtered out by the paper’s own strategies. Therefore, in the report we separately extract:

- `reservation_guard_filtered`,
- `nogood_guard_filtered`, and
- `paper_filter_filtered_total` = `reservation` + `nogood`.

This distinction is important because the total prune count reflects the entire feasibility-check framework, whereas the paper-filtered count isolates the contribution of the reproduced GuP-style filtering strategies.

4 Reproduced Paper and Reproduction Scope

The selected paper is *GuP: Fast Subgraph Matching by Guard-Based Pruning* [1]. The central insight of GuP is that many dead-end partial embeddings can be rejected before an explicit structural violation is reached. To achieve this, GuP augments the search with guard-based pruning.

4.1 Main Idea of GuP

The first mechanism is the *reservation guard*. Intuitively, when a candidate assignment (u, v) is made, some future query vertices may already be forced to consume a small set of data vertices. If one of those data vertices has already been used too early, the extension can be rejected immediately.

The second mechanism is the *nogood guard*. Instead of relying only on local consistency checks, the search records signatures of dead-end situations and uses them to reject future extensions that induce the same remaining subproblem. In the original paper, this mechanism is tightly integrated with the guarded candidate space and richer guard discovery rules.

4.2 Reproduction Scope

Table 2 summarizes the reproduced components of GuP.

The main gap from the original system lies in engineering detail. The reproduced system is sufficient for analyzing the two guard mechanisms and their effect on search-space reduction, while large performance differences on heavy workloads are still expected from the simplified candidate-space machinery and the Python implementation.

Table 2. Reproduction scope of our GuP implementation.

Original GuP component	Impl.	Our design	Faithfulness	Expected impact on results
Baseline exact search	Yes	Depth-first backtracking matcher	High	Shared correctness reference
Candidate generation	Yes	Label + degree filtering	Partial	Weaker than richer candidate-space engineering
Matching order	Yes	Order by candidate size, degree, and id	Partial	May reduce pruning strength on harder workloads
Reservation guard	Yes	Precomputed small reservation sets with Hall-style reasoning	Medium	Main source of search-space reduction in our results
Nogood guard	Yes	Conservative future-subproblem memoization	Partial	Safe but weaker than the original full mechanism
Guarded candidate space	Partial	Only the minimum structures required by the prototype	Partial	Limits peak performance and scalability
Low-level system optimization	No	Python correctness-first prototype	Low	Strongly affects wall-clock competitiveness on heavy workloads

5 Implementation Details

5.1 Implementation Overview

We implemented the project in Python with a correctness-oriented design philosophy. The codebase contains two main matchers: a baseline depth-first backtracking matcher and a *GUP-lite* matcher that reuses the same search framework but adds guard-based pruning. This shared architecture allows the effect of individual filtering strategies to be isolated cleanly in Section 8.

5.2 Graph Representation and Input Handling

All graph variants are converted into a unified in-memory structure `LabeledGraph`, which stores an adjacency dictionary for undirected edges and a label dictionary for vertex labels. The project currently supports three input formats: a toy text format (`v/e`), the official GuP example format (`.vertices`, `.edges`, and YAML query sets), and the `.graph` format used by the downloaded benchmark package.

5.3 Baseline Matcher

The baseline matcher constructs candidate sets, computes a matching order, and performs recursive backtracking. Each extension is validated by injectivity and edge-consistency checks. The baseline already records `result_mappings`, `partial_mappings`, `pruned_partial_mappings`, and `runtime_ms`, which makes it the common reference point for both the required multi-dataset experiments and the filtering-strategy ablation.

5.4 Candidate Generation and Matching Order

Candidate generation was upgraded from pure label filtering to a `label + degree` rule: a data vertex is kept as a candidate only if the labels match and the data-graph degree is at least the query-graph degree. The matching order is determined by ascending candidate-set size, descending degree, and vertex identifier as a tiebreaker. This part corresponds to the reproduced paper’s candidate-space preparation, although it remains simpler than the richer engineering used in the original GuP implementation.

5.5 Reservation Guard

The reservation guard evolved through two stages. The earlier version behaved as a lightweight runtime forward check. The improved version precomputes small reservation sets before search. For a candidate assignment (u, v) , it inspects

Table 3. Datasets used in the report.

Dataset	$ V $	$ E $	Queries used	Role
Yeast	3112	12519	dense-4: 1, 10	Main
Human	4674	86282	dense-4: 1, 10	Main
HPRD	9460	34998	dense-4: 1, 10	Main
WordNet	76853	120399	dense-4: 1; sparse-8: 15	Boundary
Patents	3774768	16518947	dense-4: 1; sparse-8: 15	Boundary

forward neighbors of u in the matching order and collects compatible future candidates. If a small subset of those future candidates satisfies a Hall-style equality between subset size and the size of the candidate union, the union is treated as reserved. At runtime, the guard only tests whether one of those reserved data vertices has already been used.

In the final experiments, this component is the main source of nonzero paper-filtered partial mappings.

5.6 Nogood Guard

The current nogood guard is a conservative memoization mechanism and does not reproduce the full GuP nogood machinery. The current version stores a signature of the future subproblem after a candidate extension, including the search depth, the hypothetical used data vertices, and the frontier mapping. Under the fixed matching order, these fields determine the remaining completion problem; the current candidate identity is deliberately omitted to allow symmetric histories to share a signature. We reduce overhead by activating signature lookups only on query vertices that have already accumulated relevant dead-end information, while still allowing an inactive vertex to learn its first failed state.

In our reproduction this component remains workload-sensitive. It contributes little on the six fixed dense-4 workloads, but the broader post-fix validation in Section 8 identifies dense-8 Yeast queries where repeated failing subproblems make nogood reuse effective.

5.7 Metrics and Experiment Scripts

All variants share the same metric object, which records counts for results, partial mappings, pruned extensions, guard checks, guard-specific prune reasons, and runtime. To support reproducible experiments, we built a small workflow around three layers: single-run scripts, batch scripts for standard ablation, and result-summarization scripts that aggregate JSON outputs into report-oriented CSV tables.

5.8 Current Limitations

The implementation still differs from the original paper in several ways: the nogood mechanism remains simplified, the guarded candidate space is not fully reconstructed, the matching order is more basic than in GuP, and the entire system is implemented in Python. The implementation nevertheless supports the main analyses in this paper: reproduction of GuP’s core mechanisms, multi-dataset evaluation with explicit metrics, and direct ablation of the two reproduced filtering strategies.

6 Experimental Setup

The benchmark data were taken from the public dataset package used in the GuP study and loaded through the project’s unified `.graph` parser. The experiments include three main datasets and two larger boundary datasets.

Table 4. Filtering-strategy switches used in the experiments.

Variant	Reservation guard	Nogood guard
Baseline	Off	Off
Reservation only	On	Off
Nogood only	Off	On
Full GUP	On	On

The main evaluation is based on three successful datasets: Yeast, Human, and HPRD. For each of these datasets, we report results on two real benchmark queries so that the report covers both lighter and heavier search regimes. WordNet and Patents are retained as time-boxed boundary datasets because the current Python prototype still reaches its scalability limit on them.

6.1 Compared Variants

We compare four variants, corresponding directly to the required filtering-strategy ablation.

6.2 Measured Metrics and Boundary Runs

For every successful run, we record runtime, result mappings, partial mappings, and paper-filtered partial mappings. The paper-filtered count is computed from the prune-reason log and currently consists of the number of extensions rejected by the reservation and nogood guards. On the main workloads, this count is dominated by the reservation guard because the current nogood implementation contributes little explicit pruning.

For the boundary datasets, each run is limited to 120 seconds. Runs that do not finish within this limit are reported as timeout.

7 Required Experimental Results on Multiple Datasets

Table 5 reports the main results on the three successful datasets. The table lists runtime, result mappings, partial mappings, and the number of partial mappings filtered by the reproduced paper’s strategies.

Table 5. Required metrics on the three successful datasets. ‘Paper-filtered PM’ denotes the number of partial mappings pruned by GuP-style filtering strategies only.

Dataset	Query	Variant	Result mappings	Partial mappings	Paper-filtered PM	Runtime (ms)
HPRD	d4-1	Baseline	980	1514	0	237.90
HPRD	d4-1	Full GUP	980	1440	74	208.63
HPRD	d4-1	Nogood only	980	1514	0	224.05
HPRD	d4-1	Reservation only	980	1440	74	211.05
HPRD	d4-10	Baseline	4	51	0	8.96
HPRD	d4-10	Full GUP	4	11	27	1.35
HPRD	d4-10	Nogood only	4	51	0	6.88
HPRD	d4-10	Reservation only	4	11	27	1.36
Human	d4-1	Baseline	10312	11013	0	185.86

Table 6. Boundary datasets under the 120-second limit.

Dataset	Query	Time limit (s)	Status
WordNet	d4-1	120	timeout
WordNet	s8-15	120	timeout
Patents	d4-1	120	timeout
Patents	s8-15	120	timeout

Dataset	Query	Variant	Result mappings	Partial mappings	Paper-filtered PM	Runtime (ms)
Human	d4-1	Full GUP	10312	10916	97	143.34
Human	d4-1	Nogood only	10312	11013	0	161.53
Human	d4-1	Reservation only	10312	10916	97	170.25
Human	d4-10	Baseline	12830306	12937898	0	50314.48
Human	d4-10	Full GUP	12830306	12937608	290	56733.61
Human	d4-10	Nogood only	12830306	12937898	0	55239.79
Human	d4-10	Reservation only	12830306	12937608	290	55887.18
Yeast	d4-1	Baseline	720	772	0	19.27
Yeast	d4-1	Full GUP	720	772	0	27.72
Yeast	d4-1	Nogood only	720	772	0	22.68
Yeast	d4-1	Reservation only	720	772	0	19.11
Yeast	d4-10	Baseline	19678	22624	0	805.98
Yeast	d4-10	Full GUP	19678	22283	341	731.28
Yeast	d4-10	Nogood only	19678	22624	0	809.12
Yeast	d4-10	Reservation only	19678	22283	341	682.42

Taken together, the three successful datasets support three main observations. First, Yeast, Human, and HPRD provide a balanced view across lighter and heavier workloads.

Second, among the reproduced paper strategies, only the reservation guard contributes nonzero paper-filtered partial mappings consistently on the real workloads considered here.

Third, the benefit of reservation-based pruning is clearly workload-dependent: it is strongest on HPRD d4-10, moderate on HPRD d4-1 and Yeast d4-10, and very small on Human d4-10.

HPRD is especially informative because it fills the gap between the smaller datasets and the boundary workloads. On HPRD d4-10, reservation-based pruning reduces the number of partial mappings from 51 to 11 and improves runtime from 8.96 ms to about 1.35 ms. On HPRD d4-1, it reduces the search space by 74 partial mappings and improves runtime by about 12%.

The results on Human and Yeast are more mixed, but still informative. On Human d4-1, full GUP achieves the best runtime among all tested variants. On Yeast d4-10, both reservation-only and full GUP improve runtime, and reservation-only gives the strongest speedup. By contrast, on Human d4-10, the reservation guard still reduces the search space slightly, but the prototype does not convert that gain into lower runtime.

Table 6 records the boundary workloads explicitly. These runs do not contribute complete metric rows, but they remain important because they mark the scalability range of the Python prototype. Together, these results show both the workloads handled well by the prototype and those that remain beyond its current boundary.

8 Ablation Study of Filtering Strategies

To assess the contribution of each filtering strategy, we disable them in turn and compare the resulting changes in runtime and partial mappings. Table 7 reports this comparison directly.

Table 7. Ablation results relative to the baseline. Negative deltas indicate improvement over the baseline.

Dataset	Query	Variant	Partial delta	Partial delta (%)	Runtime delta (ms)
HPRD	d4-1	Reservation only	-74	-4.89	-26.85
HPRD	d4-1	Nogood only	0	0.00	-13.85
HPRD	d4-1	Full GUP	-74	-4.89	-29.27
HPRD	d4-10	Reservation only	-40	-78.43	-7.60
HPRD	d4-10	Nogood only	0	0.00	-2.08
HPRD	d4-10	Full GUP	-40	-78.43	-7.61
Human	d4-1	Reservation only	-97	-0.88	-15.61
Human	d4-1	Nogood only	0	0.00	-24.33
Human	d4-1	Full GUP	-97	-0.88	-42.52
Human	d4-10	Reservation only	-290	-0.00	5572.70
Human	d4-10	Nogood only	0	0.00	4925.30
Human	d4-10	Full GUP	-290	-0.00	6419.13
Yeast	d4-1	Reservation only	0	0.00	-0.15
Yeast	d4-1	Nogood only	0	0.00	3.42
Yeast	d4-1	Full GUP	0	0.00	8.45
Yeast	d4-10	Reservation only	-341	-1.51	-123.56
Yeast	d4-10	Nogood only	0	0.00	3.14
Yeast	d4-10	Full GUP	-341	-1.51	-74.71

The ablation results support three concise conclusions.

First, the **reservation guard is the dominant effective filtering strategy** in our reproduction (Figure 1). It is responsible for every nonzero paper-filtered partial-mapping count in the main experiments and for the strongest speedups, especially on HPRD d4-10 and Yeast d4-10.

Second, the **nogood guard remains conservative in pruning power**. In the present implementation it rarely contributes explicit nogood-based pruning on the successful real workloads, which is why its partial-mapping delta is usually zero. Even so, after reducing unnecessary guard checks, the nogood-only variant is not always slower than the baseline; on some lighter workloads it is mildly beneficial in runtime even without visible search-space reduction.

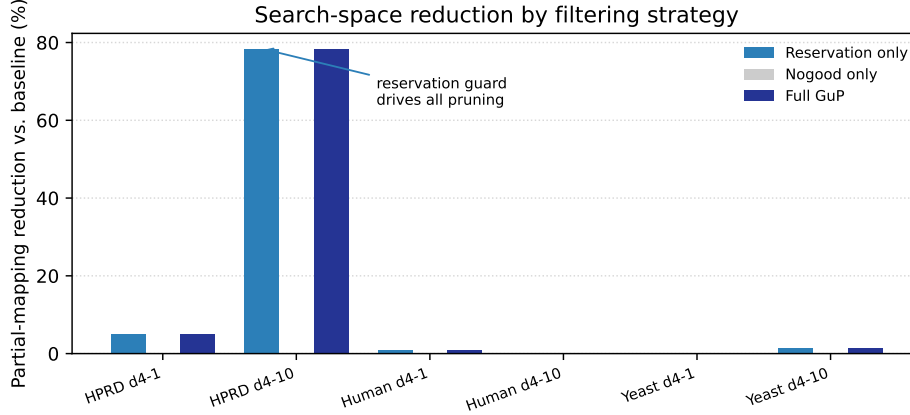


Fig. 1. Partial-mapping reduction relative to the baseline for each filtering strategy across the six real workloads. The reservation guard accounts for all measurable search-space reduction, the nogood guard contributes none on these workloads, and the effect is strongest on HPRD d4-10. Source: reports/figures/plot_ablation.py (data ablation_data.csv).

Third, the **combination of both guards is workload-dependent**. On HPRD and on Human d4-1, full GUP inherits the benefit of reservation-based pruning and outperforms the baseline. On the heavier Human d4-10, however, the prototype still pays more guard overhead than it saves. Overall, the ablation study shows that reservation-based pruning already contributes measurable value, whereas nogood-based pruning still requires a more faithful implementation to demonstrate stable gains.

8.1 Post-fix Nogood Validation on Broader Queries

After the original dense-4 ablation, we identified and fixed a cold-start issue in the Nogood implementation: signature lookup was enabled only after a query vertex became active, but the same condition previously prevented the first failed state from being learned. We also removed redundant current-candidate fields from the signature. Under the fixed matching order, the future subproblem is determined by the search depth, used data-vertex set, and frontier mapping. Correctness is checked by the full test suite and by deterministic random differential tests against the baseline.

We then scanned a broader set of real Yeast dense-8 queries and selected two completed queries with nonzero Nogood hits for repeated validation. Table 8 reports medians after warm-up with rotated execution order. This is a targeted mechanism-validation experiment rather than an unbiased replacement for the six fixed dense-4 workloads; the selection criterion is disclosed because it favors workloads where Nogood is applicable.

The two queries expose complementary behavior. On d8-103, both guards contribute and the full variant achieves the best result. On d8-132, Reservation is almost inactive, while Nogood removes about 56% of the partial mappings and more than doubles enumeration speed. Together with the fixed dense-4 results, this supports a narrower conclusion than “one guard always dominates”: Reservation is robust on the original selected workloads, whereas Nogood becomes valuable when many search histories induce the same failing future subproblem.

Table 8. Post-fix validation on real Yeast dense-8 queries. Runtime values are medians of repeated runs. All variants return identical complete result sets.

Query	Variant	Results	Partial mappings	Reservation prunes	Nogood prunes	Median (ms)	Speedup
d8-103	Baseline	38	463	0	0	59.75	1.00×
d8-103	Reservation only	38	238	221	0	28.89	2.07×
d8-103	Nogood only	38	290	0	173	37.24	1.60×
d8-103	Full GUP	38	168	221	70	19.40	3.08×
d8-132	Baseline	548	13461	0	0	3217.62	1.00×
d8-132	Reservation only	548	13452	5	0	3258.45	0.99×
d8-132	Nogood only	548	5932	0	2818	1418.07	2.27×
d8-132	Full GUP	548	5924	5	2817	1412.19	2.28×

Table 9. Result reuse for related queries with one added query edge. “Filtered” counts how many previous-query results are removed by the new edge constraint.

Dataset	Query pair	Prev results	Reuse results	Scratch results	Filtered	Reuse time (ms)	Scratch time (ms)
Toy	prev→next	2	2	2	0	0.00075	0.00546
HPRD	d4-10-e01 → d4-10	4	4	4	0	0.00250	2.68837
Human	d4-1-e01 → d4-1	10384	10312	10312	72	1.60363	66.45746

9 Optional Extension: Result Reuse for Related Queries

We also examine whether a later query can reuse the result of an earlier related query. We focus on a restricted but practically meaningful workload pattern: the data graph is fixed, and the next query Q_{next} is obtained from the previous query Q_{prev} by adding exactly one edge while keeping the vertex set and all labels unchanged.

9.1 Restricted Setting and Reuse Rule

This setting models a common query-refinement workflow: a user first evaluates a looser pattern and then tightens it by adding one missing structural constraint. In this case, every valid result mapping of Q_{next} must already be a valid result mapping of Q_{prev} . Instead of rerunning full subgraph matching from scratch, it is enough to scan the already materialized result mappings of Q_{prev} and test whether the images of the newly added query edge also form an edge in the data graph.

Formally, if Q_{next} is obtained by adding one query edge (u, v) to Q_{prev} , then

$$\text{Result}(Q_{next}) = \{f \in \text{Result}(Q_{prev}) \mid (f(u), f(v)) \in E_G\}.$$

The experiments compare the *incremental reuse cost* against the cost of evaluating Q_{next} from scratch. Since the result mappings of Q_{prev} are already available, the comparison focuses on the cost of the next query.

9.2 Optional-Extension Results

Table 9 summarizes the result-reuse experiments. We report one toy query pair and two real query pairs. In every case, the reused result set exactly matches the scratch result set.

The experiments support three observations. First, correctness is validated directly: for all three query pairs, the reused result set is identical to the scratch result set. This confirms that the added-edge filter is sound for the restricted query relation considered here.

Second, even when the new edge does not change the final result count, reuse can still be much cheaper than rerunning full matching. The HPRD pair is the clearest example: the final result count remains four before and after the added edge, but the incremental reuse cost is still far smaller than the scratch cost because the latter must rebuild the full search process for the tighter query.

Third, the Human pair shows the clearest workload-level benefit. The looser query returns 10,384 result mappings, and the added edge filters out 72 of them, leaving exactly the 10,312 mappings found by scratch evaluation. More importantly, the incremental reuse step takes only about 1.60 ms, compared with about 66.46 ms for scratch evaluation of the refined query. This result suggests that query-result reuse can substantially accelerate interactive query refinement when related queries are evaluated on the same data graph.

The current extension only supports a very restricted relation between successive queries. It does not yet handle vertex additions, arbitrary query rewrites, or cross-query reuse of partial mappings and nogood information. Even within this narrow setting, the experiments show that result reuse can be a practical workload-level optimization.

10 Optional Extension: Incremental Result Maintenance under Single-Edge Updates

We also study how to maintain the set of result mappings when the *data graph* changes, rather than the query. We consider the most basic update granularity required by the assignment: the query Q is fixed, and the data graph changes by inserting or deleting exactly one undirected edge $e = (a, b)$. The goal is twofold: (i) compute the new result set without recomputing it from scratch, and (ii) report the exact *result difference*, that is, which mappings are added or removed by the update.

10.1 Locality and Monotonicity of the Update

Our incremental algorithm rests on a simple but decisive structural property. Because a mapping f is a valid embedding if and only if every query edge is mapped to a data edge, the result set depends on the data edge set only through edge preservation. Toggling a single edge $e = (a, b)$ therefore changes the validity of an embedding f *if and only if* f relies on e .

Let G be the graph before the update, G' the graph after, and let $H \in \{G, G'\}$ be whichever version *contains* e . Define the *affected set*

$$D_e = \{ f \in \text{Result}(Q, H) \mid \exists (u, v) \in E_Q, \{f(u), f(v)\} = \{a, b\} \}.$$

Then the result difference is exactly D_e , and the update is monotone:

$$\text{insertion: } \text{Result}(Q, G') = \text{Result}(Q, G) \cup D_e \text{ (disjoint),} \quad \text{deletion: } \text{Result}(Q, G') = \text{Result}(Q, G) \setminus D_e.$$

The argument is direct. Any embedding that does not use e maps every query edge to a data edge that is present in both G and G' , so its validity is unchanged. An embedding that uses e maps some query edge to (a, b) or (b, a) ; it is valid precisely in the version that contains e . Hence an insertion can only *add* mappings (those in D_e , which were impossible before), a deletion can only *remove* mappings (those in D_e , which become invalid), and no other mapping is affected.

10.2 Anchored-Enumeration Algorithm

The property above reduces incremental maintenance to computing D_e , which we obtain by an *anchored* search. For every query edge $(u, v) \in E_Q$ and every orientation that is label-compatible with e , we pin the partial assignment $f(u) = a, f(v) = b$ (or the reverse) and run a constrained backtracking search over the remaining query vertices in

Table 10. Incremental single-edge maintenance vs. from-scratch recomputation. “Anchored PM” is the number of partial mappings the incremental search explores. Speedup is scratch runtime divided by incremental runtime.

Dataset	Update	Prev results	New results	Added	Removed	Anchored PM	Incr. (ms)	Scratch (ms)	Speedup
Toy	insert	0	2	2	0	2	0.0137	0.0038	0.27×
Toy	delete	2	0	0	2	2	0.0149	0.0003	0.02×
Demo-stress	insert	4,314	4,320	6	0	6	2.638	688.86	261×
Demo-stress	delete	4,320	4,314	0	6	6	2.374	679.92	286×

the version of the graph that contains e . Each completed embedding is recorded; results found through more than one anchor are deduplicated. By the locality property, the union of these anchored searches is exactly D_e .

Maintenance then applies the monotone rule: on insertion the new result set is the previous set plus D_e ; on deletion it is the previous set minus D_e . Crucially, the incremental method never enumerates the full result set of the changed graph—it only explores the neighborhood reachable from the single changed edge. The reference implementation lives in `src/subgraph_match/dynamic/` and the experiment harness in `scripts/run_dynamic_experiment.py`.

10.3 Correctness Validation

Correctness is verified at two levels. First, an automated test suite (`tests/test_dynamic_maintenance.py`) performs an *exhaustive sweep*: on small graphs it deletes every existing edge and inserts every label-compatible non-edge, and checks in each case that the incrementally maintained result set is identical to a full from-scratch recomputation. Second, every experiment below independently recomputes the result on the changed graph and confirms set equality of the new result, the added set, and the removed set. All reported runs satisfy `correctness_match = True`.

10.4 Experimental Results

Table 10 reports incremental maintenance against from-scratch recomputation on a fixed query under one edge update. “Toy” is the small debugging graph; “Demo-stress” is a deterministic synthetic graph (`scripts/make_dynamic_demo_graph.py`) on which the path query $A-B-C$ has 4,320 matches but any single $A-B$ edge participates in exactly 6 of them (the per-block C fan-out)—exactly the regime where locality pays off. Runtimes are single-run wall-clock measurements that exclude graph loading, consistent with the rest of this report.

The results support three observations. First, correctness holds in every case: the incrementally maintained result set, including the precise added and removed mappings, matches the from-scratch recomputation. Second, the benefit is governed by locality. On the toy graph the incremental method is actually marginally *slower*, because its fixed setup cost (candidate construction and anchor enumeration) dominates when the whole graph has only two matches; this marks the small-input overhead floor. Third, once the result set is non-trivial the advantage becomes decisive (Figure 2): on the demo-stress graph the incremental method explores only 6 partial mappings and finishes in about 2.5 ms, whereas recomputing the full result set (all 4,320 mappings) takes roughly 0.68–0.69 s, a speedup of more than two orders of magnitude. This matches the locality property: the cost of maintenance scales with the size of the affected set D_e , not with the size of the full result.

The same harness accepts the real `.graph` workloads used in the main experiments via `--input-format graph`, so the table can be extended to Yeast, Human, and HPRD once those datasets are placed under `data/raw/`. This extension only supports single-edge updates on a fixed query; batched updates, vertex insertion and deletion, and reuse of the

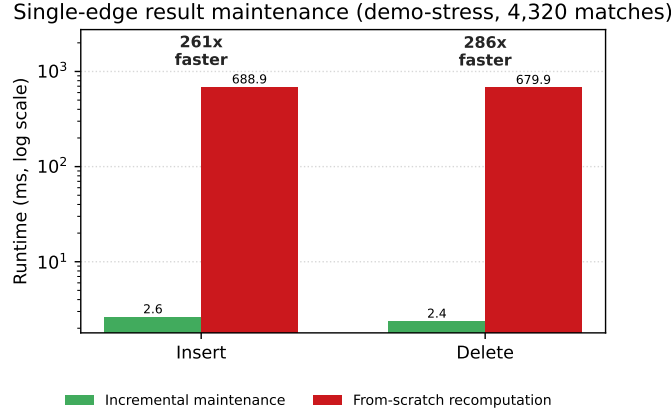


Fig. 2. Incremental single-edge maintenance vs. from-scratch recomputation on the demo-stress graph (log scale). Maintenance cost is governed by the small affected set, not the full result size, giving a two-orders-of-magnitude speedup. Source: reports/figures/plot_dynamic.py (data dynamic_data.csv).

recorded partial-mapping structure across successive updates are left as future work. Even in this restricted setting, the experiments show that single-edge result maintenance can be made effectively independent of the total result size.

11 Optional Extension: Subgraph Homomorphism

The final optional extension implements subgraph *homomorphism*, which relaxes the exact-matching semantics of the core reproduction in two ways. Given $G = (V_G, E_G, \ell_G)$ and $Q = (V_Q, E_Q, \ell_Q)$, a subgraph homomorphism is a map $f : V_Q \rightarrow V_G$ such that (a) labels are preserved, $\ell_Q(v) = \ell_G(f(v))$ for all v ; (b) every query edge maps into the augmented edge set $E_G^+ = E_G \cup \{(w, w) : w \in V_G\}$, that is, $(u, v) \in E_Q \Rightarrow (f(u), f(v)) \in E_G^+$; and (c) f need not be injective, so several query vertices may collapse onto the same data vertex. When endpoints collapse, the induced edge is a self-loop, which E_G^+ accepts uniformly, so no conflict arises. Concretely, a query edge (u, v) is satisfied iff $f(u) = f(v)$ or $(f(u), f(v)) \in E_G$.

11.1 Relationship to Subgraph Isomorphism

Homomorphism is a strict relaxation of the isomorphism studied in the main report, and the two are connected by a clean identity. Every subgraph isomorphism is a homomorphism, because $E_G \subseteq E_G^+$ and injectivity is a special case. Conversely, because the query graph is loop-free, any two distinct query vertices joined by an edge must receive *different* images under an injective map, so the self-loop case of E_G^+ never applies; an *injective* homomorphism therefore satisfies $(f(u), f(v)) \in E_G$ for every query edge and is exactly a subgraph isomorphism. Hence

$$\text{Iso}(Q, G) = \{f \in \text{Hom}(Q, G) : f \text{ is injective}\}, \quad \text{Iso}(Q, G) \subseteq \text{Hom}(Q, G),$$

and the additional homomorphisms are precisely the non-injective ones obtained by collapsing query vertices through self-loops. We use this identity as a built-in correctness oracle below.

Table 11. Subgraph homomorphism vs. subgraph isomorphism. “Injective Hom” is the injective subset of the homomorphisms and must equal “Iso”. “Non-inj. Hom” are the additional non-injective homomorphisms. “Hom PM” is the partial mappings explored by the homomorphism search.

Dataset	Query	Hom	Iso	Injective Hom	Non-inj. Hom	Hom PM	Hom (ms)	Inj. Hom = Iso
Toy	$A-A$	4	2	2	2	6	0.009	yes
Hom-demo	$A-A-A$	3,240	1,800	1,800	1,440	3,870	12.32	yes

11.2 Algorithm

The homomorphism matcher reuses the backtracking skeleton of the core reproduction with two principled changes. First, the injectivity check is removed: a data vertex may be assigned to more than one query vertex. Second, the edge-feasibility test is evaluated over E_G^+ : when extending the current partial map with $f(q) = d$, for every already-mapped query neighbor n the pair $(d, f(n))$ is accepted if $d = f(n)$ (a self-loop) or $(d, f(n)) \in E_G$.

A subtle but important consequence is that the label-and-degree (LDF) candidate filter used for isomorphism becomes *unsound* for homomorphism. Because several query neighbors may map to the same data vertex—or to $f(q)$ itself via a self-loop—a low-degree or even isolated data vertex can legitimately host a high-degree query vertex. The matcher therefore uses *label-only* candidates. The reference implementation is `src/subgraph_match/homomorphism/` with the experiment harness in `scripts/run_homomorphism_experiment.py`.

11.3 Correctness Validation

Correctness is verified at three levels in `tests/test_homomorphism_matcher.py`. First, the matcher output is compared against an independent brute-force oracle that enumerates *all* label-respecting functions $V_Q \rightarrow V_G$ and tests the E_G^+ edge constraint directly, on the toy graph and on small cliques. Second, the identity above is checked: the injective subset of the homomorphism results equals the isomorphism results returned by the baseline matcher, and the isomorphisms form a subset of the homomorphisms. Third, edge cases are covered, including collapsing two non-adjacent vertices through a self-loop and mapping an entire triangle query onto a single isolated data vertex (which an unsound degree filter would wrongly reject). All experiments below also report these cross-checks.

11.4 Experimental Results

Table 11 reports homomorphism counts against isomorphism counts. “Toy” uses the $A-A$ edge query on the debugging graph. “Hom-demo” is a deterministic synthetic graph (`scripts/make_homomorphism_demo_graph.py`) consisting of 15 disjoint cliques of size 6, all labeled A , queried with the triangle $A-A-A$; per clique every one of the 6^3 vertex assignments is a homomorphism while only $6 \cdot 5 \cdot 4$ are isomorphisms.

The results confirm the theory (Figure 3). In every case the injective subset of the homomorphisms equals the isomorphisms exactly (2 on the toy graph, 1,800 on the demo), validating both the homomorphism matcher and, independently, the baseline isomorphism matcher. The relaxation is far from cosmetic: on the toy graph the two non-injective homomorphisms collapse the $A-A$ query onto a single A vertex through the self-loop, and on the demo the self-loop relaxation produces 1,440 additional homomorphisms on top of the 1,800 isomorphisms ($3,240 = 15 \cdot 6^3$ total). Because homomorphism enumerates strictly more mappings and cannot use degree pruning, its search is somewhat heavier than the isomorphism search (12.32 ms vs. the baseline’s 11.14 ms on the demo), which is the expected cost of the broader solution space. This extension is restricted to vertex-labeled, loop-free queries and reuses the same

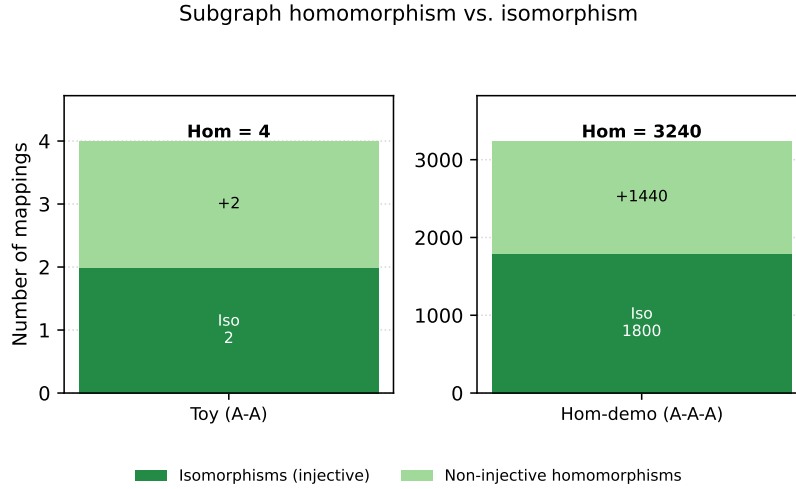


Fig. 3. Homomorphism counts decompose into the injective part (which equals the isomorphism count) and the additional non-injective homomorphisms produced by the E_G^+ self-loop relaxation. Source: reports/figures/plot_homomorphism.py (data homomorphism_data.csv).

label-only candidate model throughout; combining it with the guard-based pruning of the core reproduction is left as future work.

12 Conclusion and Reflection

This study examined the core idea of GuP through a correctness-oriented implementation for exact subgraph matching. The paper presents a literature taxonomy, a clear reproduction-scope statement, experiments on three successful datasets, a filtering-strategy ablation study, and a small extension on result reuse for related queries.

From the perspective of the development history of subgraph matching, the field has evolved from earlier candidate-space and access-method ideas [3] toward increasingly integrated systems that combine filtering, order optimization, conflict reuse, and execution engineering [2, 5, 6]. GuP is an instructive modern point in this development line because it emphasizes dynamic, state-aware pruning while remaining conceptually decomposable into interpretable modules.

Our results lead to four main conclusions. First, the experiments on Yeast, Human, and HPRD cover both lighter and heavier search regimes. Second, reservation-based pruning contributes the most consistent reduction on the six fixed dense-4 workloads, while post-fix dense-8 validation shows that nogood reuse can provide substantial gains on queries with repeated failing future subproblems. Third, the current Python prototype still has a visible scalability boundary, as shown by the boundary runs on WordNet and Patents. Fourth, under a restricted related-query setting, previously materialized result mappings can be reused exactly and can substantially reduce the incremental cost of query refinement.

One clear lesson from the study is that paper reproduction involves more than matching a final table of metrics. It also requires identifying which algorithmic ideas are central, which engineering details control practical performance, and which parts of the original system are simplified in a prototype. The largest gap between GuP and our implementation

comes from the guarded candidate-space machinery and the efficiency of the nogood mechanism. This gap helps explain the difference between a sound algorithmic idea and a highly optimized system implementation.

Overall, the study makes the core behavior of GuP visible on real workloads. Reservation and Nogood exhibit different applicability regions: Reservation is more robust on the originally selected queries, whereas Nogood can dominate when equivalent failing subproblems recur. The current results also make clear where a more faithful and efficient implementation would matter most. The result-reuse extension further suggests that workload-level reuse is a promising complementary direction: later queries in a refinement sequence can benefit from results that have already been computed.

13 Division of Labor

The project was completed by a two-person team.

- **Yi Zha:** framework construction, algorithm reproduction, end-to-end pipeline integration, and report writing.
- **Jinpeng Wang:** algorithm optimization, experimental execution, and presentation slide preparation.

Both members jointly participated in the final presentation and oral explanation of the project.

References

- [1] Junya Arai, Yasuhiro Fujiwara, and Makoto Onizuka. 2023. GuP: Fast Subgraph Matching by Guard-Based Pruning. *Proceedings of the ACM on Management of Data* 1, 2, Article 167 (June 2023), 26 pages. doi:10.1145/3589312
- [2] Minseok Han, Hwanjo Kim, Gyeong Gu, Kunsoo Park, and Wook-Shin Han. 2019. Efficient Subgraph Matching: Harmonizing Dynamic Programming, Adaptive Matching Order, and Failing Set Together. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1429–1446. doi:10.1145/3299869.3319880
- [3] Huahai He and Ambuj K. Singh. 2008. Graphs-at-a-Time: Query Language and Access Methods for Graph Databases. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data (SIGMOD '08)*. Association for Computing Machinery, New York, NY, USA, 405–418. doi:10.1145/1376616.1376655
- [4] Hyunjoon Kim, Yunyoung Choi, Kunsoo Park, Xuemin Lin, Seok-Hee Hong, and Wook-Shin Han. 2021. Versatile Equivalences: Speeding up Subgraph Query Processing and Subgraph Matching. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 925–937. doi:10.1145/3448016.3457265
- [5] Shixuan Sun and Qiong Luo. 2020. In-Memory Subgraph Matching: An In-Depth Study. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 1083–1098. doi:10.1145/3318464.3380581
- [6] Shixuan Sun, Xibo Sun, Yulin Che, Qiong Luo, and Bingsheng He. 2020. RapidMatch: A Holistic Approach to Subgraph Query Processing. *Proceedings of the VLDB Endowment* 14, 2 (2020), 176–188. doi:10.14778/3425879.3425888